Exercise 1 – Python Programming

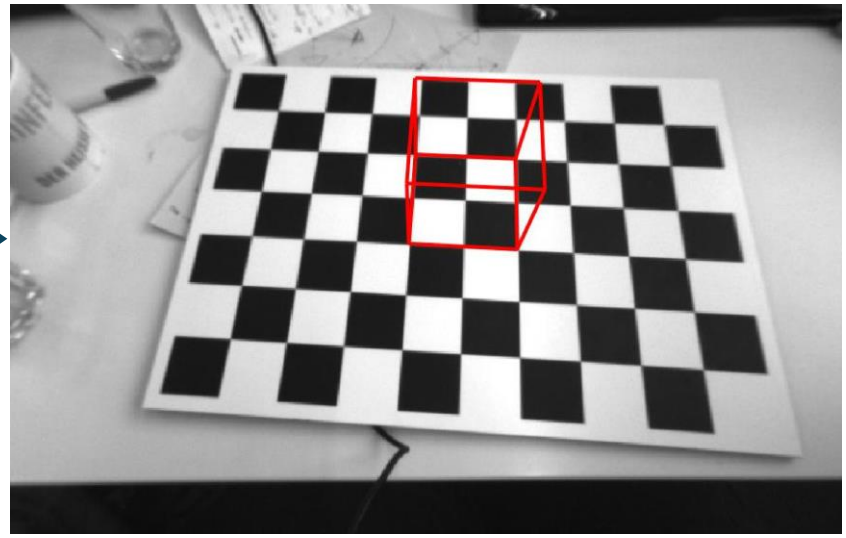
Các nội dung triển khai:

- Vẽ khối hình lập phương trên hình ảnh không bị méo
- Tính toán đến hiệu ứng biến dạng ảnh do ống kính

1.1. Vẽ hình lập phương



Ảnh gốc



Ảnh mong muốn

Các package cần thiết:

- **`numpy`**: xử lý, tính toán vector – ma trận
- **`cv2`**: đọc ảnh và xử lý ảnh (opencv)
- **`matplotlib.pyplot`**: hiển thị ảnh, vẽ đồ thị

1.1. Vẽ hình lập phương

Các bước tiến hành:

- Bước 1: Đọc ảnh, lấy dữ liệu cho trước
- Bước 2: Biểu diễn các điểm ảnh mốc
- Bước 3: Biểu diễn khối lập phương trên ảnh

1.1. Vẽ hình lập phương

❑ **Bước 1: Đọc ảnh, lấy dữ liệu cho trước**

Cú pháp:

```
img_gray = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)
```

Tên file ảnh

Chuyển sang dạng ảnh xám



1.1. Vẽ hình lập phương

❑ **Bước 1: Đọc ảnh, lấy dữ liệu cho trước**

Cú pháp:

```
K = np.loadtxt(filename_K)
```

Tên file text chứa ma trận K

```
[[420.506712   0.         355.208298]
 [  0.       420.61094    250.336787]
 [  0.         0.           1.      ]]
```

1.1. Vẽ hình lập phương

- **Bước 1: Đọc ảnh, lấy dữ liệu cho trước**

Cú pháp:

```
poses = np.loadtxt(filename_poses)
```



```
[[-0.37248319  0.03970225  0.06503934 -0.10703586 -0.14706524  0.3985125 ]
 [-0.37186741  0.03828641  0.05952308 -0.1092644  -0.14575711  0.39827766]
 [-0.3759689   0.03779668  0.05630992 -0.11072365 -0.14356467  0.39869642]
 ...
 [-0.37547236 -0.00235632  0.04262216 -0.11832042 -0.15806932  0.40173759]
 [-0.37178805  0.00330254  0.04485711 -0.11493525 -0.15950779  0.40130036]
 [-0.36765618  0.00705707  0.04608305 -0.11247752 -0.16065131  0.40107407]]
```

1.1. Vẽ hình lập phương

- **Bước 2: Biểu diễn các điểm ảnh mốc**

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \times [R|t] \times \begin{bmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{bmatrix}$$

Vị trí trong ảnh u – cột, v – hàng

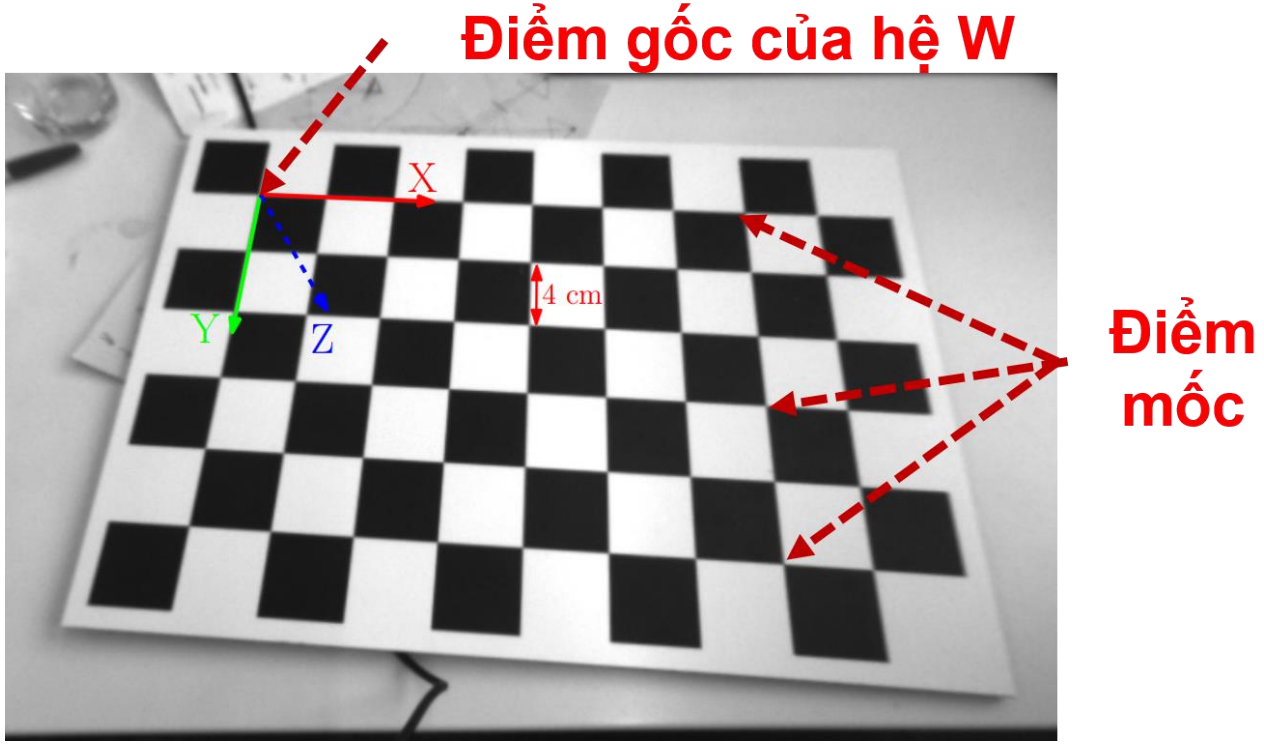
Ma trận camera (đã có)

Ma trận biến đổi W → C

Tọa độ điểm mốc trong hệ W

1.1. Vẽ hình lập phương

❑ Bước 2: Biểu diễn các điểm ảnh mốc



Chú ý:

- Các điểm mốc là các góc vuông trên bàn cờ → cách đều nhau
- Các điểm này đều nằm trên cùng mặt phẳng.
- Đưa về đơn vị mét.

1.1. Vẽ hình lập phương

❑ Bước 2: Biểu diễn các điểm ảnh mốc

Giá trị bắt đầu

Giá trị kết thúc

Số điểm chia

```
size_x = np.linspace(-1, 9, 11)
size_y = np.linspace(-1, 6, 8)
points_x, points_y = np.meshgrid(size_x, size_y)
```

Vector chứa các vị trí gốc

Hoành độ các điểm mắt lưới

Tung độ các điểm mắt lưới

Ghép các vị trí tương ứng để tạo thành tọa độ các điểm

```
[[-0.04 -0.04  0.  ]
 [ 0.   -0.04  0.  ]
 [ 0.04 -0.04  0.  ]
 [ 0.08 -0.04  0.  ]
 [ 0.12 -0.04  0.  ]
 [ 0.16 -0.04  0.  ]
 [ 0.2  -0.04  0.  ]
 [ 0.24 -0.04  0.  ]
 [ 0.28 -0.04  0.  ]
 [ 0.32 -0.04  0.  ]
```

$$\begin{bmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{bmatrix}$$

1.1. Vẽ hình lập phương

❑ Bước 2: Biểu diễn các điểm ảnh mốc

```
[[-0.37248319  0.03970225  0.06503934 -0.10703586 -0.14706524  0.3985125 ]
```

Vector góc quay $\omega = [\omega_x \quad \omega_y \quad \omega_z]$

Vector chuyển vị $t = [t_x \quad t_y \quad t_z]$

$\theta = ||\omega||$, chuẩn bậc hai của vector → **`np.linalg.norm()`**

$$\boldsymbol{k} = \frac{\omega}{\theta}$$

$$R = I + \sin\theta \times [k] + (1 - \cos\theta)[k]^2$$

$$[k] = \begin{bmatrix} 0 & -k_z & k_y \\ k_z & 0 & -k_x \\ -k_y & k_x & 0 \end{bmatrix}$$

Ma trận đơn vị → **`np.eye()`**

1.1. Vẽ hình lập phương

- **Bước 2: Biểu diễn các điểm ảnh mốc**

$$[R|t]$$

```
[[ 0.99713161 -0.07078929  0.02678482 -0.10703586]
 [ 0.05617815  0.92937006  0.36484971 -0.14706524]
 [-0.05072046 -0.36229845  0.93068108  0.3985125 ]]
```

1.1. Vẽ hình lập phương

❑ Bước 2: Biểu diễn các điểm ảnh mốc

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \times [R|t] \times \begin{bmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{bmatrix}$$

```
[[-0.04 -0.04  0.  ]
```

```
[[ 0.99713161 -0.07078929  0.02678482 -0.10703586]
 [ 0.05617815  0.92937006  0.36484971 -0.14706524]
 [-0.05072046 -0.36229845  0.93068108  0.3985125 ]]
```

```
[[420.506712   0.       355.208298]
 [  0.       420.61094  250.336787]
 [  0.         0.         1.      ]]
```

```
[ 86.83263036  25.45954706   0.41503325]
```

Chia cho phần tử cuối cùng

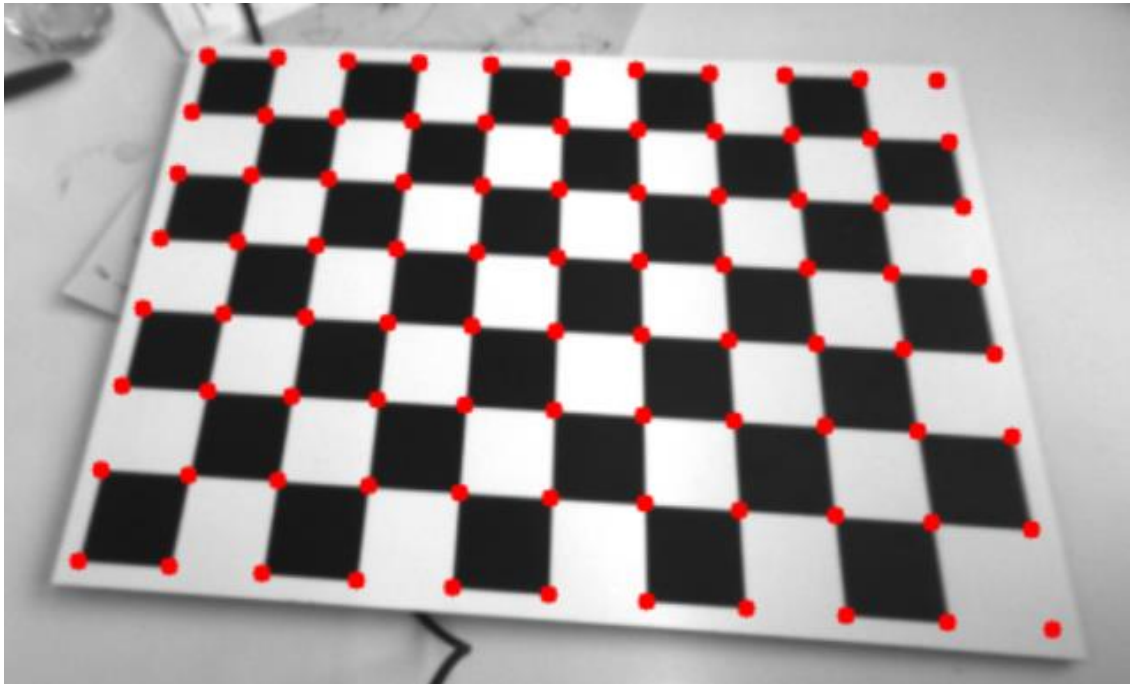
```
[209.21848881  61.34339062]
```

Làm tròn giá trị

```
[209  61]
```

1.1. Vẽ hình lập phương

- **Bước 2: Biểu diễn các điểm ảnh mốc**



```
cv2.circle(img, points[i, :], 5, (0, 0, 255), -1)
```

Tọa độ điểm (côt, hàng)

Bán kính

Màu đỏ

Tô kín

1.1. Vẽ hình lập phương

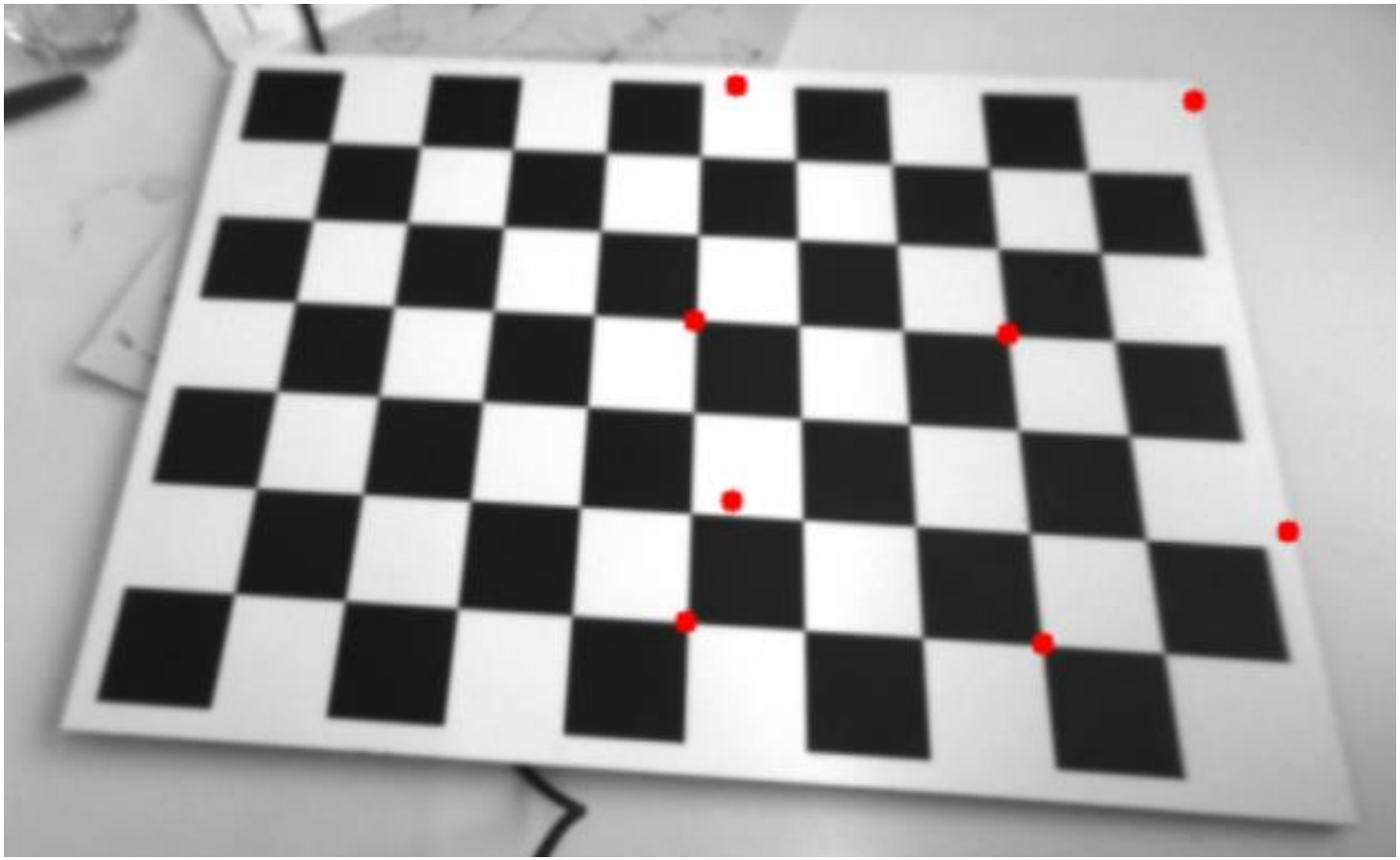
- **Bước 3: Biểu diễn khối lập phương trên ảnh**

  Bao gồm các bước sau:

  - Bước 3.1: Xác định tọa độ các đỉnh của hình lập phương trong hệ W.
  - Bước 3.2: Xác định tọa độ của các đỉnh này trong hệ tọa độ hình ảnh.
  - Bước 3.3: Vẽ các đường thẳng nối các đỉnh này với nhau để tạo thành khối lập phương.

1.1. Vẽ hình lập phương

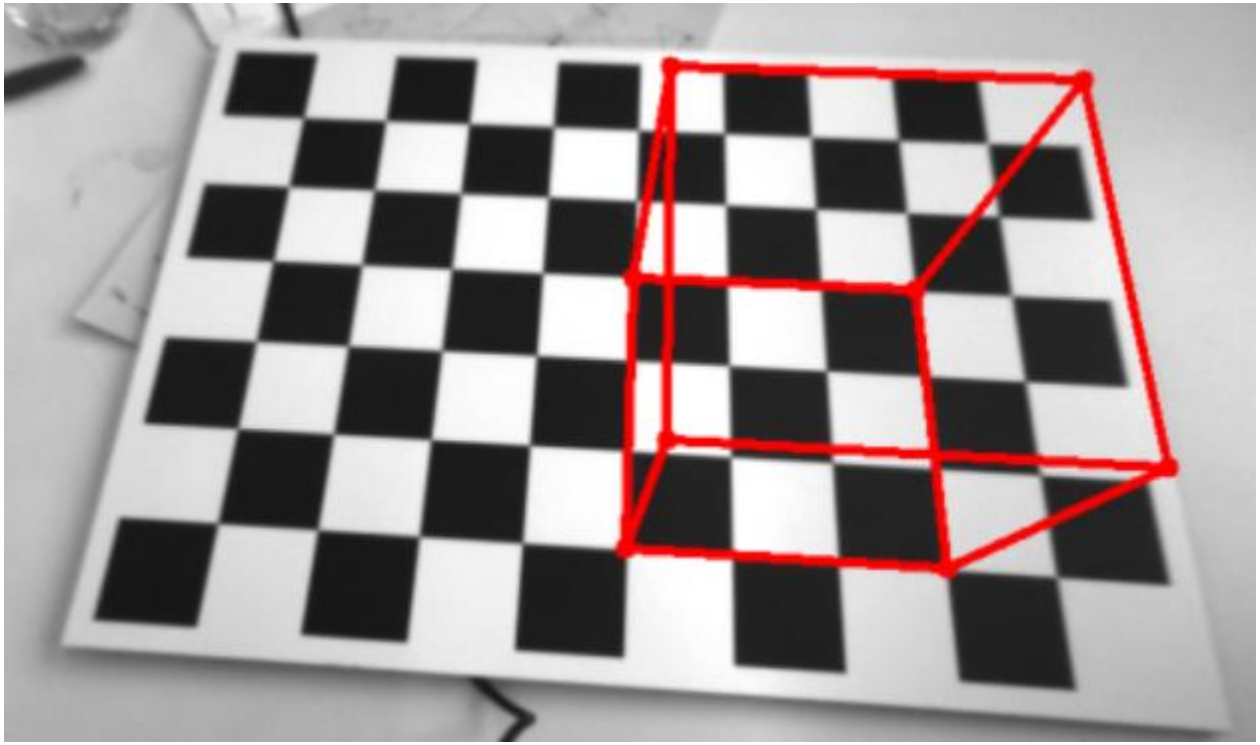
- **Bước 3: Biểu diễn khối lập phương trên ảnh**



Biểu diễn các đỉnh hình lập phương trên hình vẽ

1.1. Vẽ hình lập phương

- **Bước 3: Biểu diễn khối lập phương trên ảnh**



```
cv2.line(img, point1, point2, (0, 0, 255), thickness=3)
```

Tọa độ điểm 1 (côt, hàng)

Tọa độ điểm 2 (côt, hàng)

Màu đỏ

Độ dày

1.2. Tính toán hiệu ứng biến dạng ảnh

Các nội dung triển khai:

- Biểu diễn các điểm mốc trên ảnh bị biến dạng
- Chuyển ảnh bị biến dạng về ảnh không biến dạng

1.2. Tính toán hiệu ứng biến dạng ảnh

1.2.1. Biểu diễn các điểm mốc trên ảnh bị biến dạng

```
[[-0.04 -0.04  0.  ]
 [ 0.   -0.04  0.  ]
 [ 0.04 -0.04  0.  ]
 [ 0.08 -0.04  0.  ]
 [ 0.12 -0.04  0.  ]
```

Tọa độ trong ảnh **không** bị biến dạng

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \times [R|t] \times \begin{bmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{bmatrix}$$

$$r^2 = (u - u_0)^2 + (v - v_0)^2$$

Tọa độ trong ảnh bị biến dạng

$$\begin{bmatrix} u_d \\ v_d \end{bmatrix} = (1 + k_1 r^2 + k_2 r^4) \begin{bmatrix} u - u_0 \\ v - v_0 \end{bmatrix} + \begin{bmatrix} u_0 \\ v_0 \end{bmatrix}$$

```
[-1.6774e-06   2.5847e-12]
```

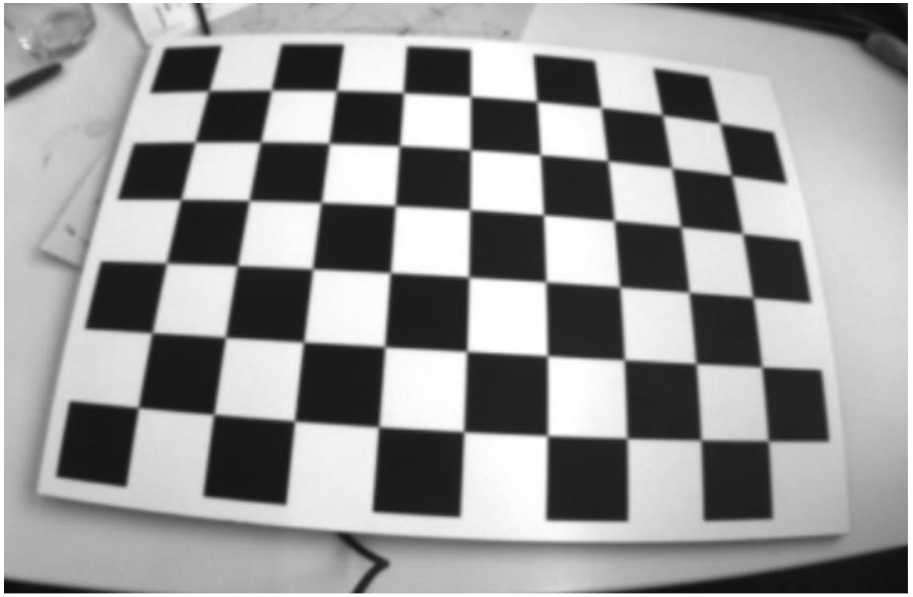
Ma trận D

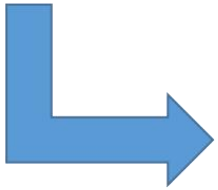
```
[[420.506712   0.         355.208298]
 [  0.       420.61094    250.336787]
 [  0.         0.           1.      ]]
```

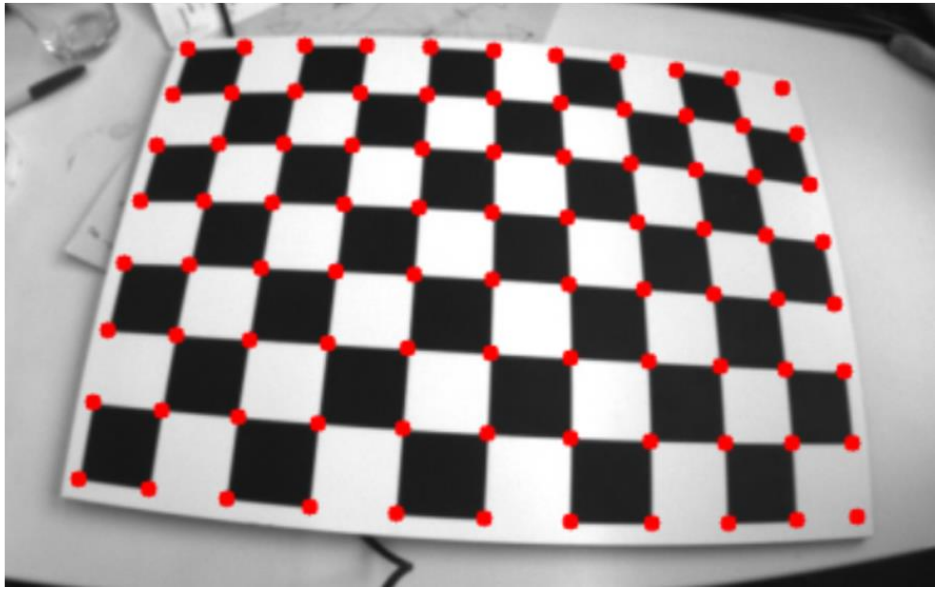
Ma trận K

1.2. Tính toán hiệu ứng biến dạng ảnh

1.2.1. Biểu diễn các điểm mốc trên ảnh bị biến dạng

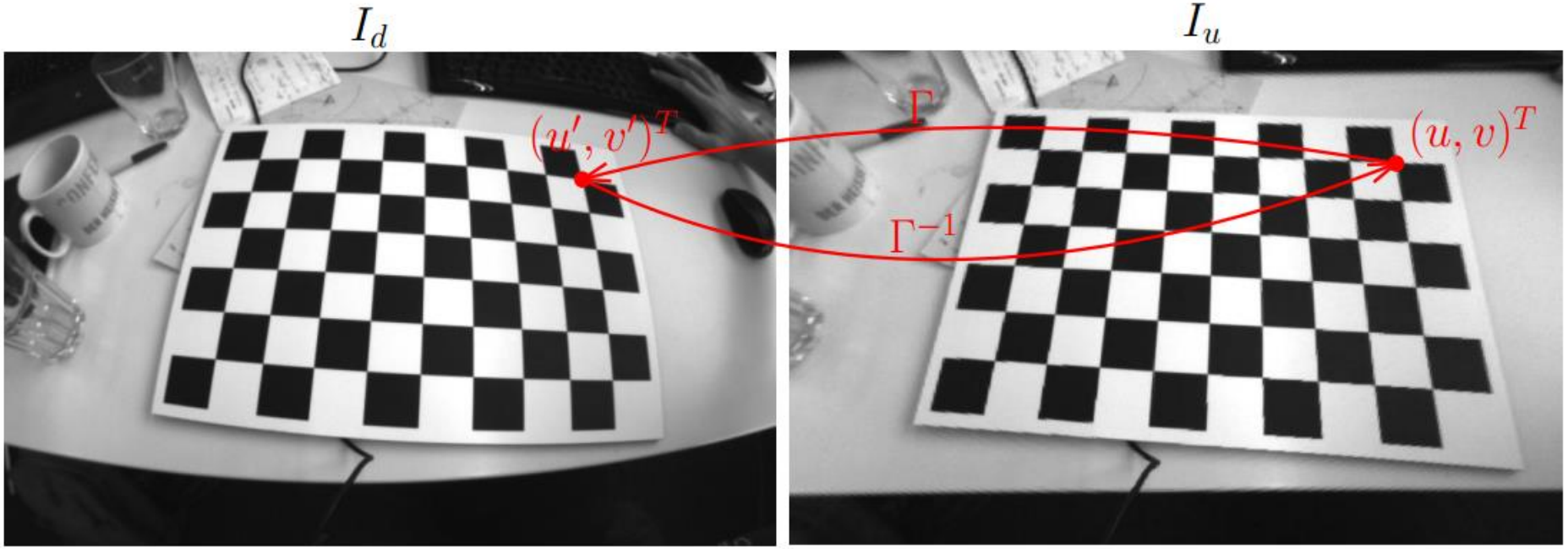






1.2. Tính toán hiệu ứng biến dạng ảnh

1.2.2. Chuyển ảnh biến dạng về ảnh không biến dạng



- Γ: I_u (ảnh không biến dạng) $\rightarrow$ I_d (ảnh biến dạng) đã được triển khai trong mục 1.2.1
- Γ^{-1}: $\mathrm{I_d} \rightarrow \mathrm{I_u}$ khó giải quyết

1.2. Tính toán hiệu ứng biến dạng ảnh

1.2.2. Chuyển ảnh biến dạng về ảnh không biến dạng

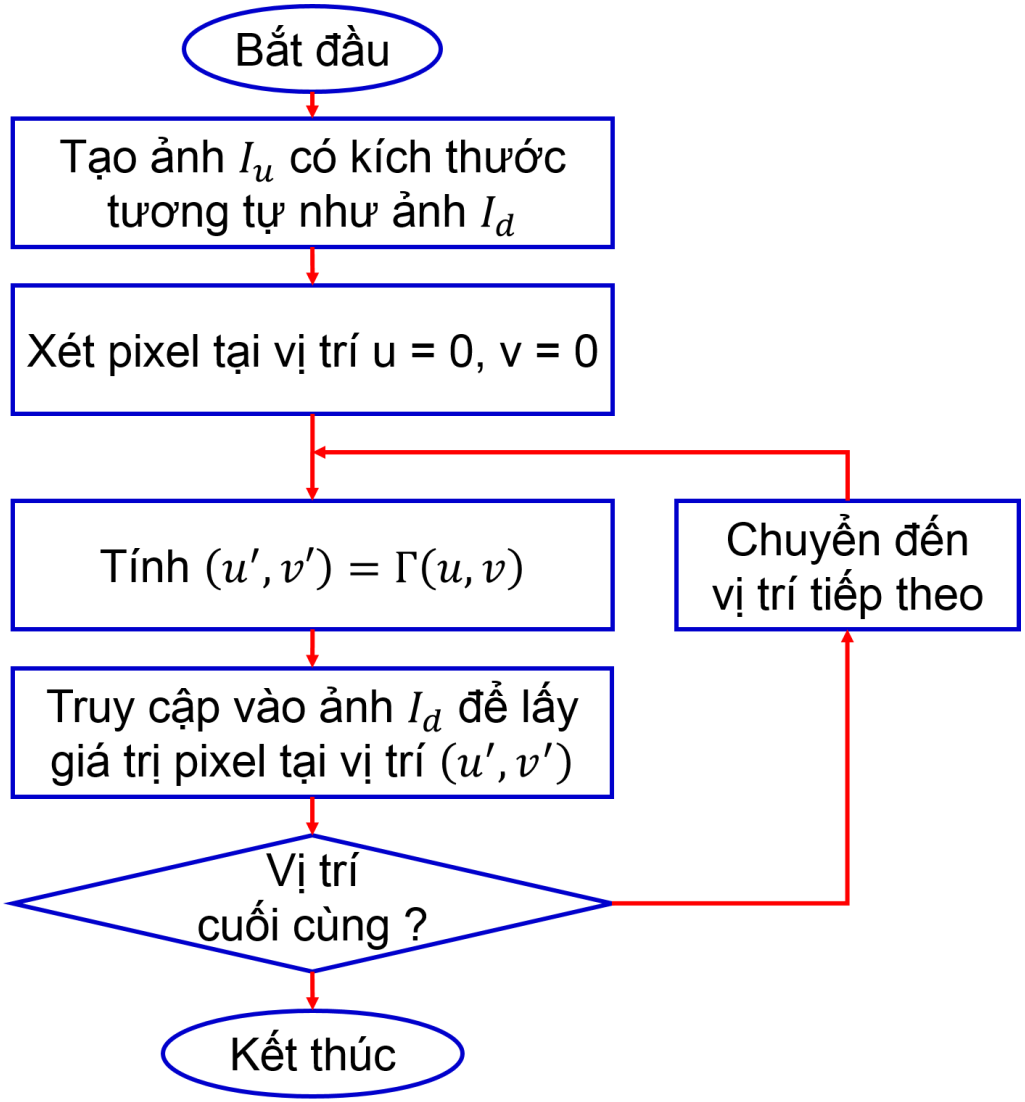
Ý tưởng:

$$I_u(u,v) = I_d\big(\Gamma(u,v)\big)$$

→ đi tìm giá trị pixel tại vị trí (u,v) của ảnh I_u thông qua giá trị pixel tại vị trí $(u',v') = \Gamma(u,v)$ của ảnh I_d

1.2. Tính toán hiệu ứng biến dạng ảnh

1.2.2. Chuyển ảnh biến dạng về ảnh không biến dạng



1.2. Tính toán hiệu ứng biến dạng ảnh

1.2.2. Chuyển ảnh biến dạng về ảnh không biến dạng

- Giá trị tọa độ (u', v') trong phép biến đổi:

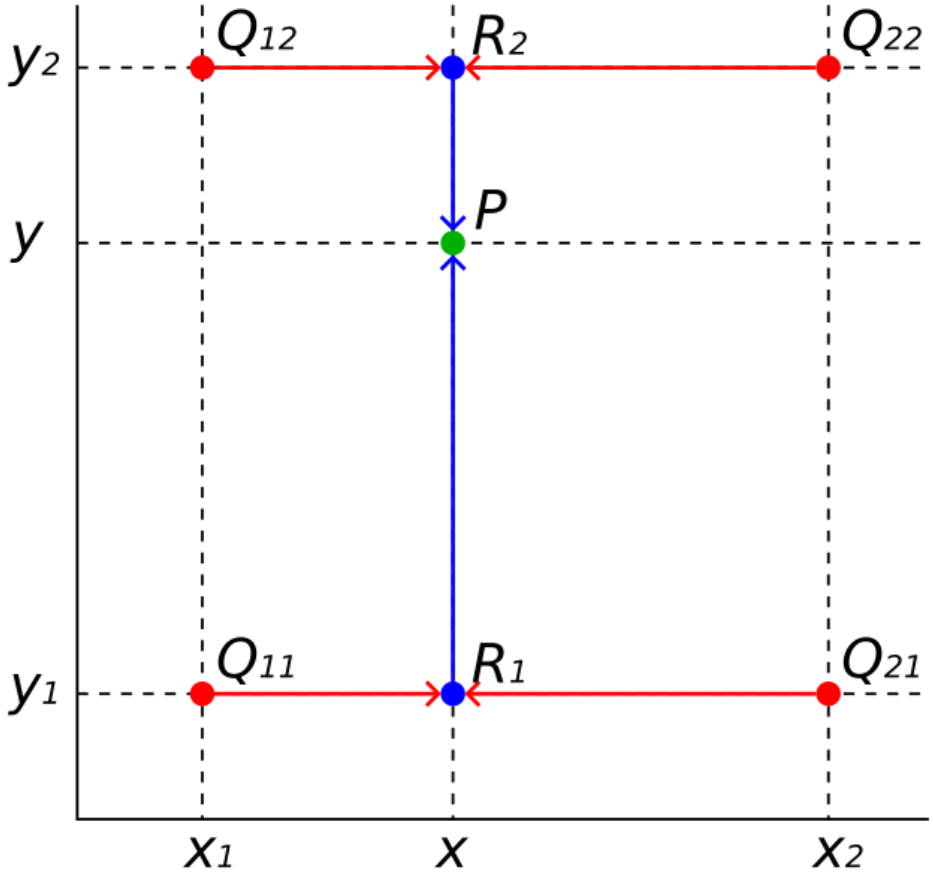
$$(u', v') = \Gamma(u, v)$$

thông thường là các giá trị thực, tuy nhiên trong hệ tọa độ ảnh, các giá trị này là các số nguyên.

- Do đó, để lấy được giá trị $I_d(u', v')$ có hai cách:
  - Làm tròn (u', v') về giá trị nguyên gần nhất (nearest-neighbor interpolation)
  - Sử dụng nội suy song tuyến (Bilinear Interpolation)

1.2. Tính toán hiệu ứng biến dạng ảnh

Nội suy song tuyến (Bilinear Interpolation)



Xét $x \in [x_1, x_2]$, $y \in [y_1, y_2]$

Đã biết các giá trị $f(Q_{11})$, $f(Q_{12})$, $f(Q_{21})$, $f(Q_{22})$

Giá trị $f(x, y)$ tính theo công thức sau

$$\frac{1}{(x_2 - x_1)(y_2 - y_1)} \begin{bmatrix} x_2 - x & x - x_1 \end{bmatrix} \begin{bmatrix} f(Q_{11}) & f(Q_{12}) \\ f(Q_{21}) & f(Q_{22}) \end{bmatrix} \begin{bmatrix} y_2 - y \\ y - y_1 \end{bmatrix}$$

1.2. Tính toán hiệu ứng biến dạng ảnh

Kết quả lập trình



Nearest-neighbor interpolation



Bilinear Interpolation